

İşletim Sistemleri

Bölüm 2: Süreçler, Thread'ler & Zamanlama Bölüm Sonu Soruları — Tam Çözümler

Kaynak	Bölüm	Kapsam	Dil
Modern Operating Systems (Tanenbaum, 4th ed.)	Chapter 2	Processes → Threads → Sync → Scheduling	TR açıklama / EN terim



Bu belgede Bölüm 2'nin Süreçler, Thread'ler, Süreçlerarası İletişim (IPC), Senkronizasyon ve Zamanlama (Scheduling) konularına ait sorular ayrıntılı Türkçe çözümlerle ele alınmıştır. Anahtar terimler her iki dilde verilmiştir.

İçindekiler

- A. Süreçler ve Durum Geçişleri (Processes & State Transitions) ([S1-2](#), [S5-8](#), [S36](#))
- B. Kesmeler ve Yığın Yönetimi (Interrupts & Stack Management) ([S3-4](#))
- C. Thread'ler — Kavramlar ve Performans ([S9-18](#))
- D. Yarış Koşulları ve Senkronizasyon (Race Conditions & Sync) ([S19-34](#))
- E. Zamanlama (Scheduling) — Algoritmalar ve Hesaplamalar ([S35](#), [S37-52](#))
- F. Açık Uçlu Quiz Soruları ve Cevapları ([—](#))

A — Süreçler ve Durum Geçişleri (Processes & State Transitions)

Soru 1 — Eksik Durum Geçişleri (Missing State Transitions)

[EN] In Fig. 2-2, three process states are shown. With three states, there could be six transitions but only four are shown. Are there circumstances in which the missing transitions might occur?

[TR] Şekil 2-2'de üç süreç durumu ve dört geçiş vardır. Eksik iki geçiş hiç gerçekleşebilir mi?

Durum	Açıklama	Mümkün mü?
Hazır (Ready)	CPU bekleyen süreç	—
Çalışıyor (Running)	CPU'da çalışan süreç	—
Bloke (Blocked/Waiting)	G/Ç veya olay bekleyen süreç	—

Dört standart geçiş:

- ① Hazır → Çalışıyor (zamanlayıcı seçti)
- ② Çalışıyor → Hazır (time-slice bitti / preempt)
- ③ Çalışıyor → Bloke (G/Ç isteği)
- ④ Bloke → Hazır (G/Ç tamamlandı)

Eksik geçiş ①: Hazır → Bloke

Normalde anlamsız — süreç CPU'ya bile girmeden neden bloke olur? Ancak sinyal (signal) veya OS tarafından zorla askıya alma (suspend) gibi nadir senaryolarda teorik olarak mümkündür.



Eksik geçiş ②: Bloke → Çalışıyor

Yalnızca tek işlemcili, tek süreçli bir sistemde mantıklı olurdu. Standart OS'lerde zamanlayıcı her zaman hazır kuyruğundan seçer; bloke bir süreç doğrudan CPU alamaz.

Process State	Süreç durumu — Ready/Running/Blocked
State Transition	Durum geçişi — olaylarla tetiklenir
Preemption	Zorla kesme — zamanlayıcı CPU'yu geri alır

Soru 2 — Donanımla Süreç Değişimi (Hardware Process Switching)

[EN] Suppose you design an advanced computer that does process switching in hardware instead of having interrupts. What information would the CPU need? Describe how it might work.

[TR] Donanım destekli süreç değişimi nasıl çalışır? CPU hangi bilgilere ihtiyaç duyar?

Donanımlı süreç değişimi için CPU, her sürecin tam bağlamını (context) kayıt altına almalıdır. Gerekli bilgiler:

Bilgi / Information	Açıklama
Program Counter (PC)	Bir sonraki çalıştırılacak komutun adresi
Register seti (Registers)	Tüm genel amaçlı ve özel amaçlı yazmaçlar
Stack Pointer (SP)	Mevcut yığın tepesinin adresi
Memory Map / Page Table Base	Sürecin adres alanını tanımlayan bilgi
Process ID (PID)	Hangi sürecin bağlamına geçileceği
Priority / Time Slice bilgisi	Zamanlama kararı için



Çalışma mekanizması: Donanım periyodik olarak (örn. her N saat döngüsünde) mevcut sürecin bağlamını bellekteki PCB (Process Control Block) yapısına kaydeder, zamanlama tablosundan bir sonraki süreci seçer ve onun bağlamını yükler. Yazılım interrupt handler'ı gereksiz kalır → bağlam değişimi çok daha hızlı olur.

Soru 5 — CPU İsrafı — 4 Süreç, %50 G/Ç Bekleme (CPU Waste with 4 Processes)

[EN] A computer system has enough room to hold four programs in its main memory. These programs are idle waiting for I/O half the time. What fraction of the CPU time is wasted?

[TR] 4 program, her biri zamanın yarısında G/Ç bekliyor. CPU'nun ne kadarı boşa gidiyor?

Her program için G/Ç bekleme oranı $p = 0.50$

Çoklu programlama formülü:

CPU kullanımı $= 1 - p^n$

n = bellekteki program sayısı $= 4$

CPU kullanımı $= 1 - (0.50)^4 = 1 - 0.0625 = 0.9375 \rightarrow \%93.75$

Boşa harcanan CPU oranı $= 1 - 0.9375 = 0.0625 \rightarrow \%6.25$



4 program ile CPU'nun yalnızca %6.25'i boşa harcanır. Çoklu programlama, G/Ç bekleyen süreçlerin yarattığı boşlukları diğer süreçlerle doldurur CPU verimliliğini dramatik biçimde artırır.

Soru 6 — Maksimum G/Ç Bekleme — %99 CPU Kullanımı (Max I/O Wait for 99% Utilization)

[EN] A computer has 2 GB of RAM, OS occupies 256 MB. Processes are all 128 MB. Goal: 99% CPU utilization. What is the maximum I/O wait that can be tolerated?

[TR] 2 GB RAM, OS 256 MB, süreçler 128 MB. %99 CPU kullanımı için maksimum G/Ç bekleme?

Kullanılabilir RAM = 2048 - 256 = 1792 MB

Süreç sayısı = 1792 / 128 = 14 süreç → n = 14

Hedef: CPU kullanımı ≥ 0.99

$1 - p^{14} \geq 0.99$

$p^{14} \leq 0.01$

$p \leq 0.01^{(1/14)}$

$p \leq 10^{(-2/14)}$

$p \leq 10^{(-0.1429)}$

$p \leq 0.7197 \rightarrow$ yaklaşık %72

Sonuç: Süreçler zamanlarının en fazla %72'sini G/Ç bekleyerek geçirebilir.

✓ 14 süreçle %99 CPU kullanımı için her sürecin zamanının en fazla ~%72'sini G/Ç bekleyerek geçirmesi tolere edilebilir. Bu, bellekte yeterince süreç bulundurmanın ne kadar kritik olduğunu gösterir.

Soru 7 — Paralel vs. Sıralı İş Tamamlama (Parallel vs Sequential Jobs)

[EN] Two jobs, each needing 10 minutes of CPU time, start simultaneously. How long to complete if run sequentially? In parallel? Assume 50% I/O wait.

[TR] 2 iş, her biri 10 dk CPU zamanı, %50 G/Ç bekleme. Sıralı vs. paralel süre?

Her işin toplam süresi: CPU zamanı / (1 - G/Ç bekleme oranı)

= 10 dakika / (1 - 0.50) = 20 dakika (gerçek duvar saati süresi)

SIRALI (Sequential):

İş 1 biter: 20 dk

İş 2 biter: 20 + 20 = 40 dakika

Son tamamlanma = 40 dakika

PARALEL (Parallel / Multiprogrammed):

Her iki iş de CPU'yu paylaşır.

Efektif CPU payı = $1 - (0.5)^2 = 0.75 \rightarrow$ her iş CPU'nun %50'sini alır

Her iş için toplam süre $\approx 10 / 0.50 = 20$ dakika (eş zamanlı koşarken)

Son tamamlanma ≈ 20 dakika

Paralel çalışma, toplam süreyi 40 dk'dan 20 dk'ya indirir.

✓ Paralel çalışma 2x hızlıdır. İşler G/Ç beklerken diğeri CPU'yu kullanır. Her iki iş de 20 dakikada tamamlanır; sıralıda ise 40 dakika gerekir.

Soru 8 — Çoklu Programlama CPU Kullanımı — 5 Süreç, %40 G/Ç Bekleme

[EN] Consider a multiprogrammed system with degree of 5 (five programs in memory). Each process spends 40% of its time waiting for I/O. What is the CPU utilization?

[TR] 5 programlı sistemde her süreç %40 G/Ç beklerse CPU kullanımı nedir?

```
p = 0.40 (G/Ç bekleme oranı)
n = 5 (bellekteki program sayısı)

CPU kullanımı = 1 - p^n
= 1 - (0.40)^5
= 1 - 0.01024
= 0.98976
≈ %98.98
```

✓ 5 süreç ve %40 G/Ç bekleme oranıyla CPU kullanımı ~%99'a ulaşır. $p^n = 0.01024$, yani tüm programların aynı anda G/Ç beklediği ihtimal yalnızca %1'dir. Çoklu programlama bu senaryoda son derece verimlidir.

CPU Utilization	CPU kullanım oranı — $1 - p^n$ formülü
Degree of Multiprogramming	Bellekteki eşzamanlı program sayısı
I/O Wait (Blocking Probability)	G/Ç bekleme olasılığı = p

Soru 36 — fork() ile Oluşturulan Alt Süreç Sayısı (Child Processes from fork)

[EN] void main(){ fork(); fork(); exit(); } How many child processes are created?

[TR] İki ardışık fork() çağırısı kaç alt süreç oluşturur?

```
Başlangıç: 1 süreç (P0)

1. fork() çağırısı:
P0 → P0 ve P1 oluşur (toplam: 2 süreç)

2. fork() çağırısı: Her iki süreç de fork() yapar:
P0 → P0 + P2
P1 → P1 + P3
(toplam: 4 süreç)
```

Oluşturulan ALT süreçler: P1, P2, P3 → 3 alt süreç
(P0 zaten vardı, o sayılmaz)

★ N adet fork() çağrısından $2^N - 1$ alt süreç oluşur. 2 fork() → $2^2 - 1 = 3$ alt süreç.

B — Kesmeler ve Yığın Yönetimi (Interrupts & Stack Management)

Soru 3 — Kesme İşleyicisi Neden Assembly? (Why Interrupt Handlers in Assembly?)

[EN] On all current computers, at least part of the interrupt handlers are written in assembly language. Why?

[TR] Kesme işleyicilerinin en azından bir kısmı neden assembly dilinde yazılır?

Neden Assembly?	Açıklama
Yazmaç (Register) Erişimi	Tüm yazmaçları kaydetmek/geri yüklemek için doğrudan donanım erişimi gerekir. Yüksek seviyeli diller bu erişimi garanti etmez.
Yığın Değişimi (Stack Switch)	Kesme sırasında kullanıcı yığınının çekirdek yığına geçiş yapılmalıdır; bu işlem C derleyicisi tarafından otomatik yapılamaz.
Atomik / Kritik Sekans	Belirli komut dizilerinin kesintisiz (atomic) çalışması gerekir; derleyici optimizasyonları bu sırayı bozabilir.
Özel Komutlar	CLI/STI (interrupt enable/disable), IRET gibi ayrıcalıklı komutlar yalnızca assembly ile kullanılabilir.

i

Modern OS'ler (Linux, Windows) minimum assembly kullanır — yalnızca donanıma özgü kritik kısımlar assembly; geri kalanı C ile yazılır.

Soru 4 — Neden Ayrı Kernel Yığını? (Why a Separate Kernel Stack?)

[EN] When an interrupt or system call transfers control to the OS, a kernel stack area separate from the stack of the interrupted process is generally used. Why?

[TR] Kesme/sistem çağrısında neden sürecin kendi yığını değil, ayrı bir kernel yığını kullanılır?

Neden Ayrı Yığın?	Açıklama
Güvenlik (Security)	Kullanıcı süreci kendi yığını kontrol eder. Kernel verilerini bu yığına yazmak stack smashing / privilege escalation saldırısına kapı açar.

Bellek Koruma (Memory Protection)	Kullanıcı adres alanı ile kernel adres alanı ayrıdır. Kernel, kullanıcı yığınınına erişemeyebilir (farklı sayfa tablosu).
Süreci Bulmak (Process Identification)	Kernel yığını, hangi sürecin kernel içinde olduğunu belirsizlik yaratmadan tanımlar. Her süreç için ayrı kernel yığını alanı mevcuttur.
Güvenilirlik (Reliability)	Kullanıcı programı yığınını bozsa bile kernel yığını etkilenmez; sistem kararlılığını korur.
Kernel Stack	Her süreç için kernel modunda kullanılan ayrı yığın
User Stack	Kullanıcı modunda çalışan programın yığını
Stack Smashing	Yığın taşması saldırısı — güvenlik açığı

C — Thread'ler (Threads) — Kavramlar ve Performans

Soru 9 — Web Tarayıcısında Thread Kullanımı (Browser Threading)

[EN] Explain how a Web browser can utilize the concept of threads to improve performance.

[TR] Bir web tarayıcısı performansı artırmak için thread'leri nasıl kullanabilir?

Thread Görevi	Açıklama
Ana UI Thread'i	Kullanıcı etkileşimlerini (tıklama, kaydırma) işler; donmaları önler.
Ağ Thread'i (Network Thread)	HTTP isteklerini gönderir, yanıtları alır. Ana UI thread bloke olmaz.
Render Thread'i	HTML/CSS ayrıştırma ve sayfa çizimi yapar; ağ işlemlerinden bağımsız.
JavaScript Engine Thread	JS kodunu çalıştırır; ağır hesaplamalar diğer işlemleri bloke etmez.
I/O Thread'leri	Resim, video, font indirme işlemlerini paralel yürütür.



Somut örnek: Bir sayfa yüklenirken UI thread'i butonlara tıklamaya yanıt verirken, ağ thread'i 10 farklı kaynağı paralel indirir; render thread'i ise gelen içeriği anında çizer. Tek thread ile bu işlemler sıralı olurdu.

Soru 10 — Paralel İndirme — Thread ile Hızlandırma (Parallel Download with Threads)

[EN] You are trying to download a large 2-GB file from mirror servers, each delivering a subset of bytes. Explain how you might use threads to improve the download time.

[TR] 2 GB dosyayı ayna sunuculardan thread'ler kullanarak nasıl hızlı indirirsiniz?

Her thread, dosyanın farklı bir byte aralığını farklı bir ayna sunucudan ister:

2 GB = 2.048 MB → N adet thread, N adet ayna sunucu

Thread 1: GET /file bytes=0 - 524287 (0 - 512 MB)

Thread 2: GET /file bytes=524288 - 1048575 (512 MB - 1 GB)

Thread 3: GET /file bytes=1048576- 1572863 (1 GB - 1.5 GB)

Thread 4: GET /file bytes=1572864- 2097151 (1.5 GB - 2 GB)

Ana thread tüm parçaları bekler (join) ve birleştirir.

İdeal hızlanma: $N \times$ tek sunucu bant genişliği

i Bu tekniğe segmented / parallel download denir. BitTorrent, aria2c, wget --split gibi araçlar bunu kullanır. Ağ gecikmeleri nedeniyle hızlanma $N \times$ 'den az olur ama belirgin iyileşme sağlanır.

Soru 11 — Dosya Sunucusu ve Thread Modeli (File Server Thread Model)

[EN] In the text, the model of Fig. 2-10(a) was not suited to a file server using a cache in memory. Why not? Could each process have its own cache?

[TR] Tek thread'li model neden bellek içi önbellek kullanan dosya sunucusu için uygun değil?

x Neden uygun değil? Şekil 2-10(a)'daki model (tek thread, finite-state machine) tüm istekleri sıralı işler. Bir blok önbellekte (cache) bulunmuyorsa disk okuması yapılır ve tüm sistem bu disk G/Ç'sini bekler. Thread yoksa diğer istekler de bekler. Önbellek varlığından faydalanmak için paralel işleme gerekir.

⚠ Her süreç kendi önbelleğini tutabilir mi? Evet, ama verimsiz olur. Aynı dosya bloğu N farklı süreç tarafından N kez önbelleklenir — bellek israfı. Paylaşılan tek önbellek (shared cache) ile thread modeli çok daha verimlidir: bir thread önbelleğe yüklerse diğerleri de aynı bloğu kullanır.

Soru 12 — Web Sunucusu — Kullanıcı mı Kernel Thread mi? (User vs Kernel Threads)

[EN] If the only way to read a file is the normal blocking read system call, do you think user-level or kernel-level threads are being used for the Web server? Why?

[TR] Yalnızca bloke eden read() varsa, web sunucusu kullanıcı mı kernel düzeyinde thread mi kullanır?

Thread Türü	Bloke Eden read() ile Ne Olur?
Kullanıcı Düzeyi Thread (User-Level Thread)	Kullanıcı thread'i bloke eden read() yaparsa TÜM süreç bloke olur. Kernel hangi thread'in beklediğini bilmez. → Uygun DEĞİL.
Kernel Düzeyi Thread (Kernel-Level Thread)	Yalnızca ilgili thread bloke olur. Kernel diğer thread'leri çalıştırmaya devam eder. → Dosya sunucusu için UYGUN.

✓ Cevap: Kernel düzeyi thread'ler kullanılmalıdır. Kullanıcı düzeyi thread'ler bloke eden sistem çağrılarıyla birlikte tüm süreci dondurur; yüksek performanslı dosya sunucusu için bu kabul edilemez.

Soru 13 — Tek Thread'li Sunucunun Avantajı (When Single-Thread Server Wins)

[EN] Are there any circumstances in which a single-threaded server might be better? Give an example.

[TR] Tek thread'li sunucunun çok thread'li sunucuya üstün olabileceği durumlar var mı?

Durum	Neden Tek Thread Daha İyi?
Tüm yanıtlar önbellekte (All cache hits)	Disk G/Ç olmadığına thread değişim (context switch) maliyeti gereksizdir. Tek thread minimum gecikmeyle yanıt verir.
Basit, düşük yük sunucu (Low-traffic server)	Thread senkronizasyonu ve oluşturma maliyeti, kazançtan fazla olabilir.
Hata ayıklama (Debugging)	Tek thread deterministik davranır; yarış koşulu (race condition) yoktur. Geliştirme/test aşamasında idealdir.
Deterministik işleme (Deterministic processing)	İstek sırası önemliyse (FIFO garanti) tek thread doğal çözümdür.

Soru 14 — Kayıt Seti Neden Thread'e Özgü? (Why Per-Thread Register Set?)

[EN] In Fig. 2-11 the register set is listed as a per-thread rather than per-process item. Why? After all, the machine has only one set of registers.

[TR] Makine tek yazmaç setine sahipken neden kayıt seti süreç değil thread başına tutulur?

i Açıklama: Fiziksel olarak tek yazmaç seti vardır, ancak her thread ayrı bir yürütme akışına (execution context) sahiptir. Thread değişimi sırasında mevcut thread'in yazmaçları bellekteki TCB'ye (Thread Control Block) kaydedilir, yeni thread'in yazmaçları geri yüklenir. Bu nedenle yazmaç seti mantıksal olarak thread başına özeldir; süreçteki tüm thread'ler aynı bellek alanını paylaşırsa da her birinin farklı PC, SP ve genel yazmaç değerleri vardır.

TCB (Thread Control Block)	Thread bağlamını tutan veri yapısı
Execution Context	Yürütme bağlamı — PC, SP, yazmaçlar
Context Switch	Bağlam değişimi — yazmaç kaydet/yükle

Soru 15 — Thread Neden Gönüllü CPU Bırakır? (Why thread_yield?)

[EN] Why would a thread ever voluntarily give up the CPU by calling thread_yield? After all, since there is no periodic clock interrupt, it may never get the CPU back.

[TR] Periyodik saat kesmesi olmadan neden thread_yield çağrılır?

Neden thread_yield()?	Açıklama
İş birlikli zamanlama (Cooperative Scheduling)	Kullanıcı düzeyi thread'ler kernel kesmesine sahip değildir. Thread_yield(), diğer thread'lerin ilerlemesine izin verir.
Kaynak beklerken (Waiting for Resource)	Thread ihtiyacı olan kaynağı (kilit, veri) bulamazsa busy-wait yerine CPU'yu bırakmak diğer thread'lerin kaynağı serbest bırakmasını sağlar.
Uzun döngüler (Long Loops)	Hesapca ağır bir döngüde periyodik olarak yield çağırarak diğer thread'lerin aç kalmasını (starvation) önler.



CPU geri alınması: Kullanıcı düzeyi thread sisteminde run-time scheduler thread'leri sırayla seçer. thread_yield() kuyruğa geri koyar → diğerleri çalışır → sıra tekrar gelir. Clock interrupt yoksa bu tek yol.

Soru 16 — Tek vs. Çok Thread'li Dosya Sunucusu — Verim Hesabı (Server Throughput)

[EN] 15 msec to get a request and process it (cache hit). If disk is needed (1/3 of requests), 75 msec extra (thread sleeps). How many requests/sec for single-threaded and multi-threaded server?

[TR] Tek thread'li ve çok thread'li sunucu kaç istek/saniye işler?

— TEK THREAD'Lİ SUNUCU —

Ortalama istek süresi:

$$= (2/3 \times 15 \text{ ms}) + (1/3 \times (15 + 75) \text{ ms})$$

$$= (2/3 \times 15) + (1/3 \times 90)$$

$$= 10 + 30 = 40 \text{ ms}$$

$$\text{Verim} = 1000 \text{ ms} / 40 \text{ ms} = 25 \text{ istek/saniye}$$

— ÇOK THREAD'Lİ SUNUCU —

Disk G/Ç sırasında thread uyur, başka thread çalışır.

CPU meşgul süresi = 15 ms (disk bekleme CPU'yu bloke ETMEZ)

Verim = 1000 ms / 15 ms $\approx$ 66.7 istek/saniye

Hızlanma = 66.7 / 25 $\approx$ 2.67x

- ✓ Çok thread'li sunucu ~2.67 kat daha fazla istek işler. Disk G/Ç beklerken thread uyuduğundan CPU boşa gitmez; diğer thread'ler bu sürede hizmet verir.

Soru 17 — Kullanıcı Düzeyi Thread — En Büyük Avantaj ve Dezavantaj

[EN] What is the biggest advantage of implementing threads in user space? What is the biggest disadvantage?

[TR] Thread'leri kullanıcı alanında uygulamanın en büyük avantajı ve dezavantajı nedir?

	Açıklama
✓ En Büyük Avantaj (Biggest Advantage)	Hız (Speed): Thread oluşturma ve değişimi tamamen kullanıcı alanında gerçekleşir; sistem çağrısı (kernel geçişi) gerekmez. Kullanıcı düzeyi bağlam değişimi ~10x daha hızlıdır.
✗ En Büyük Dezavantaj (Biggest Disadvantage)	Bloke eden sistem çağrısı: Bir thread bloke eden read() yaparsa tüm süreç bloke olur. Kernel hangi thread'in beklediğini bilmez. Ayrıca gerçek paralel CPU kullanımı (multicore) mümkün değildir.

Soru 18 — Thread Sırası Kontrolü (Forcing Thread Order)

[EN] Is there a way to force the order: thread1 created → prints → exits, then thread2 created → prints → exits, etc.?

[TR] Thread oluşturma ve yazdırma sırası zorunlu kılınabilir mi?

Evet, senkronizasyon mekanizmaları ile mümkündür:

- Semafor veya mutex kullanarak her thread, bir öncekinin tamamlandığını belirten sinyal bekler.
- i • pthread_join() ile her thread oluşturulduktan hemen sonra tamamlanması beklenir, ancak bu thread'leri sıralı çalıştırır — paralel avantajı ortadan kalkar.
- Koşul değişkenleri (condition variables) ile thread1 tamamlayınca thread2 oluşturulabilir.

```
// Semafor ile sıralı thread çalıştırma (örnek)
```

```
sem_t sem;  
sem_init(&sem, 0, 0);  
  
void* thread_func(void* arg) {  
    int id = *(int*)arg;  
    printf('Thread %d\n', id);  
}
```

```
sem_post(&sem); // bir sonrakini serbest bırak
return NULL;
}

// Ana süreçte: thread'leri tek tek oluştur ve bekle
for (int i = 0; i < N; i++) {
pthread_create(&tid, NULL, thread_func, &i);
sem_wait(&sem); // bu thread bitmeden devam etme
}
```

D — Yarış Koşulları ve Senkronizasyon (Race Conditions & Synchronization)

Soru 19 — Yarış Koşulunu Tanımlama — get_account (Identify Race Condition)

[EN] Identify a race condition in the get_account function where two threads share 'accounts' array and 'account_count' global.

[TR] İki thread aynı accounts[] ve account_count değişkenini kullanırsa hangi yarış koşulu oluşur?

```
// Kritik bölge (Critical Section) – yarış koşulu burada:
accounts[account_count] = entered_account;
account_count++;

// Senaryo: T1 ve T2 eşzamanlı çalışıyor
T1: account_count'u oku → 5
T2: account_count'u oku → 5 (T1 henüz yazmadı!)
T1: accounts[5] = X; account_count = 6
T2: accounts[5] = Y; account_count = 6 // X kayboldu!

Sonuç: Aynı indekse iki yazma → veri kaybı (data loss)
account_count yanlış → bir slot eksik sayılabilir
```



Yarış koşulu (Race Condition): Paylaşılan değişkenlere atomik olmayan erişim. Düzeltme için: mutex ile kritik bölgeyi koruyun (pthread_mutex_lock/unlock).

Race Condition	Yarış koşulu — sonuç yürütme sırasına bağlı
Critical Section	Kritik bölge — aynı anda tek thread erişmeli
Atomicity	Atomiklik — bölünmez işlem garantisi

Soru 20 — Thread'lerde Global Değişken — Pointer mı Değer mi? (Global Variables: Pointer vs Value)

[EN] In the discussion on global variables in threads, we used `create_global` to allocate storage for a pointer to the variable, rather than the variable itself. Is this essential?

[TR] Değişkenin kendisi yerine neden pointer'ına depolama alanı tahsis edilir?



Evet, pointer kullanmak gereklidir. Thread'e özgü (thread-local) veri için: bir thread'in kendi kopyasına erişmesi pointer aritmetiği veya indeks ile yapılır. Değerin kendisi global alanda tutulsaydı tüm thread'ler aynı değeri paylaşırdı — bu thread-local storage (TLS) amacını ortadan kaldırır. Pointer ise her thread'in kendi ayrı bellek alanına işaret eder.

Soru 21 — Saat Kesmesi Sırasında Kullanıcı Thread Zamanlayıcısı (Clock Interrupt in User Thread)

[EN] Threads in user space, clock interrupt once a second. What problem might occur if the interrupt comes while the run-time system is blocking/unblocking a thread?

[TR] Kesme tam thread kuyruğunu değiştirirken gelirse ne olur? Çözümü nedir?



Problem: Kullanıcı düzeyi zamanlayıcı, hazır kuyruğunu güncellerken saat kesmesi gelirse run-time sistem veri yapıları tutarsız hale gelebilir (örn. yarım eklenmiş thread). Bu çekirdeğin kritik bölgesini kesmesiyle aynıdır.



Çözüm: Run-time sistem kritik bölgeye girerken kesmeler geçici olarak devre dışı bırakılır (disabled). Kullanıcı alanında bu POSIX sinyal maskeleyme (sigprocmask) ile yapılabilir; kritik bölge bitince kesmeler yeniden etkinleştirilir.

Soru 22 — `select()` Olmadan Non-blocking Thread Paketi (User Threads without `select`)

[EN] OS has no `select()` but allows alarm clocks that interrupt blocked system calls. Can we implement a user-space thread package that won't block all threads?

[TR] `select()` yoksa alarm saat kesmesiyle tüm thread'leri bloke etmeden paket kurulabilir mi?



Evet, kısmen mümkündür. Algoritma:

- 1 Bloke eden çağrı yapmadan önce alarm set et (örn. 1 ms sonra).
 - 2 Bloke eden çağrı yapılır (read, recv...).
 - 3 Alarm süresi dolunca SIGALRM sinyali gelir → çağrı EINTR ile kesilir.
 - 4 Signal handler başka thread'i çalıştırır.
 - 5 Orijinal çağrı hazır olduğunda yeniden denenir (retry loop).
- Dezavantaj: Polling'e benzer → CPU israfı ve gecikme.

Soru 23 — Peterson Çözümü — Preemptive / Non-preemptive (Peterson's Solution)

[EN] Does Peterson's solution to mutual-exclusion work when scheduling is preemptive? How about when it is nonpreemptive?

[TR] Peterson'ın çözümü preemptive ve non-preemptive zamanlama ile çalışır mı?

Zamanlama	Çalışır mı?	Açıklama
Preemptive (Kesintili)	✓ Evet	Peterson paylaşımlı değişkenleri (turn, interested[]) atomik olmayan işlemlerle günceller. Uniprocessor'da en kötü senaryoda bile doğruluğu korunur çünkü yalnızca bir süreç çalışır. Multiprocessor'da bellek sıralama garantileri (memory barriers) gerekebilir.
Non-preemptive (Kesintisiz)	✓ Evet	Süreç CPU'yu gönüllü bırakmadan önce kritik bölgeyi tamamlarsa başka süreç kesemez. Çalışır, ancak starvation riski daha düşüktür.



Önemli not: Peterson çözümü modern çok-çekirdekli sistemlerde tek başına yetmeyebilir; bellek sıralama (memory ordering) ve önbellek tutarlılığı (cache coherency) sorunları eklenebilir. Pratikte memory_fence/barrier eklenmelidir.

Soru 24 — Priority Inversion — Kullanıcı Thread'leri (Priority Inversion with User Threads)

[EN] Can the priority inversion problem happen with user-level threads? Why or why not?

[TR] Öncelik tersine dönme (priority inversion) sorunu kullanıcı düzeyi thread'lerde oluşabilir mi?



Hayır, klasik anlamda oluşmaz. Öncelik tersine dönme, kernel zamanlayıcısının yüksek öncelikli süreci düşük öncelikli sürecin tuttuğu kaynağı beklerken bloke etmesinden kaynaklanır. Kullanıcı düzeyi thread'lerde yalnızca kullanıcı alanı zamanlayıcısı çalışır; kernel hangi thread'in yüksek öncelikli olduğunu bilmez ve müdahale etmez. Ancak kullanıcı alanı zamanlayıcısı öncelik uyguluyor ve mutex kullanılıyorsa, benzer bir sorun teorik olarak yaşanabilir.

Soru 25 — Round-Robin'de Öncelik Açlığı Sorunu (Priority Problem with Round-Robin)

[EN] A high-priority H and low-priority L process led to H looping forever. Does the same problem occur with round-robin scheduling?

[TR] Yüksek öncelikli H sürecinin sonsuza dönmesi sorunu Round-Robin ile de olur mu?

Hayır, Round-Robin bu sorunu önler. Senaryo: L bir kaynağı tutuyor, H kaynağı bekliyor. Priority scheduling'de H sürekli CPU alır ama kaynağa erişemez → sonsuz döngü. Round-Robin'de H bir time-slice alır, ardından L de time-slice alır. L kaynağı serbest bırakır. H ilerler. Bu nedenle Round-Robin'de öncelik tersine dönme daha az zararlıdır.

Soru 26 — Thread Başına Yığın Sayısı (Stack per Thread or per Process?)

[EN] In a system with threads, is there one stack per thread or one stack per process for user-level threads? What about kernel-level threads?

[TR] Kullanıcı ve kernel düzeyi thread'lerde yığın (stack) kaç tane olur?

Thread Türü	Yığın Sayısı	Açıklama
Kullanıcı Düzeyi (User-Level Threads)	Her thread için ayrı yığın	Run-time kütüphane her thread için kullanıcı alanında ayrı yığın tahsis eder. Kernel tek yığın görür (sürecin yığın bölgesi).
Kernel Düzeyi (Kernel-Level Threads)	Her thread için ayrı yığın	Her kernel thread için hem kullanıcı yığını hem de kernel yığını mevcuttur. Kernel her birini bağımsız yönetir.

i Her iki durumda da thread başına bir yığın gerekir: fonksiyon çağrılarını, yerel değişkenleri ve dönüş adresleri yığında saklanır. Her thread'in bağımsız çağrı zinciri (call chain) vardır.

Soru 27 — Yarış Koşulu Nedir? (What is a Race Condition?)

[EN] What is a race condition?

[TR] Yarış koşulu (race condition) nedir?

◆ Tanım (Definition): İki veya daha fazla sürecin/thread'in aynı paylaşılan veriye eşzamanlı erişmesi durumunda sonucun yürütme sırasına bağlı olması. Belirsiz ve tekrar edilemeyen hatalara (non-deterministic bugs) yol açar.

```
// Klasik örnek: shared_counter++
// Makine kodu düzeyinde atomik değil:
LOAD R1, shared_counter // T1 okur: 5
LOAD R1, shared_counter // T2 okur: 5 (T1 henüz yazmadı)
ADD R1, 1 // T1: R1 = 6
ADD R1, 1 // T2: R1 = 6
STORE R1, shared_counter // T1 yazar: 6
STORE R1, shared_counter // T2 yazar: 6 ← KAYIP GÜNCELLEME!
// Beklenen: 7 Gerçek: 6 → Race Condition!
```


Soru 28 — Simülasyonda Yarış Koşulu (Race Condition in Sequential Simulation)

[EN] When a computer is being simulated sequentially (one instruction at a time), is it possible for a race condition to occur?

[TR] Bilgisayar sıralı simüle edilirken (tek komut aynı anda) yarış koşulu oluşabilir mi?



Evet, oluşabilir. Yarış koşulu için eşzamanlı fiziksel yürütme şart değil; komutların araya girmesi (interleaving) yeterlidir. Sıralı simülasyon bile iki sürecin komutlarını keyfi sırada yürütebilir. Örn: P0'ın LOAD'ı, ardından P1'in LOAD'ı, ardından P0'ın STORE'u — paylaşılan değişken tutarsız olur. Simülasyon sırası deterministik olsa da farklı araya girme (interleaving) kombinasyonları test edildiğinde yarış koşulları keşfedilebilir.

Soru 29 — Çok Üretici-Çok Tüketici — Semafor Çözümü (Multi-Producer Multi-Consumer)

[EN] The producer-consumer problem extended to multiple producers and consumers, one shared buffer. Will the semaphore solution in Fig. 2-28 work?

[TR] Birden fazla üretici-tüketici ile Fig. 2-28'deki semafor çözümü çalışır mı?



Evet, çalışır — ekstra bir mutex (mutual exclusion semaphore) ile. Standart çözümdeki empty/full semaforları üretim/tüketim sayısını kontrol eder. Ancak birden fazla üretici (veya tüketici) aynı anda tampona erişmeye çalışırsa paylaşılan tampon indeksi yarış koşuluna girer. Çözüm: tampona erişim mutex ile korunmalıdır.

```
// Gerekli semaforlar:
semaphore mutex = 1; // tampon erişim kilidi
semaphore empty = N; // boş slot sayısı
semaphore full = 0; // dolu slot sayısı

// Üretici:
down(0);
down(&mutex); // kritik bölge başlat
buffer[in] = item; in = (in+1) % N;
up(&mutex); // kritik bölge bitir
up(&full);
```

Soru 30 — Strict Alternation Çözümü Doğru mu? (Strict Alternation Correctness)

[EN] P0: while(turn!=0){} CS; turn=0; P1: while(turn!=1){} CS; turn=1. Does this meet all required conditions for mutual exclusion?

[TR] Katı sıra değişimi (strict alternation) çözümü karşılıklı dışlamayı doğru sağlar mı?

Koşul	Sağlanıyor mu?	Açıklama
Karşılıklı Dışlama (Mutual Exclusion)	✓ Evet	turn yalnızca biri için 0 ya da 1 olabilir; eşzamanlı erişim engellenir.
İlerleme (Progress)	✗ Hayır	P0 kritik bölgede değilken P1 sırası gelir ama P0 tekrar girene kadar P1 beklemek zorundadır. Yani kritik bölgeye girme sırası zorunlu.
Sınırlı Bekleme (Bounded Waiting)	✓ Evet	En fazla 1 tur beklemek yeterli.
Busy Wait	✗ Kötü	CPU'yu boşa tüketir; spin-lock davranışı.

✗ Sonuç: Çözüm karşılıklı dışlamayı sağlar ancak ilerleme (progress) koşulunu ihlal eder: P0 kritik bölgede olmasa bile P1'in girmesini engelleyebilir → doğru bir çözüm DEĞİLDİR.

Soru 31 — Sayaç Semafor → İkili Semafor ile Gerçekleme (Counting → Binary Semaphore)

[EN] Show how counting semaphores can be implemented using only binary semaphores.

[TR] Sayaç semafor, yalnızca ikili semaforlarla nasıl gerçekleştirir?

```
// Gerekli ikili semaforlar:
binary_semaphore s_mutex = 1; // sayacı korur
binary_semaphore s_wait = 0; // bekleyen süreçleri bloke eder
int count; // semafor değeri

// down(S) – P işlemi:
void down(int *count) {
    P(s_mutex);
    (*count)--;
    if (*count < 0) {
        V(s_mutex);
        P(s_wait); // bloke ol
    } else {
        V(s_mutex);
    }
}
```

```

}

// up(S) – V işlemi:
void up(int *count) {
    P(s_mutex);
    (*count)++;
    if (*count <= 0) {
        V(s_wait); // bekleyeni uyandır
    }
    V(s_mutex);
}

```

Soru 32 — İki Süreçli Sistemde Bariyer (Barrier with Two Processes)

[EN] If a system has only two processes, does it make sense to use a barrier to synchronize them?

[TR] Yalnızca iki süreç varsa bariyer (barrier) kullanmak mantıklı mıdır?

i

Evet, bazen mantıklıdır. İki sürecin belirli bir hesaplama aşamasını tamamlamasını beklemek gerekiyorsa bariyer kullanılabilir. Ancak iki süreç için bariyer, basit bir çift semafor veya join ile de sağlanabilir. Bariyer N süreç için tasarlanmıştır; iki süreç için maliyet düşük ama fayda da sınırlıdır.

Soru 33 — Kernel Semafor ile Thread Senkronizasyonu (Kernel Semaphore for Threads)

[EN] Can two threads in the same process synchronize using a kernel semaphore if threads are implemented by the kernel? What if they are implemented in user space?

[TR] Kernel ve kullanıcı düzeyi thread'lerde kernel semaforuyla senkronizasyon mümkün müdür?

Thread Türü	Kernel Semafor Çalışır mı?	Açıklama
Kernel Düzeyi Thread	✓ Evet	Kernel her thread'i tanır. Bir thread semafor üzerinde bloke olursa yalnızca o thread askıya alınır; diğeri çalışmaya devam eder.
Kullanıcı Düzeyi Thread	✗ Hayır	Kernel yalnızca süreci görür. Bir thread kernel semaforu beklerse TÜM süreç bloke olur. Senkronizasyon yalnızca kullanıcı alanı mekanizmaları ile yapılmalıdır.

Soru 34 — Mailbox Busy-Wait — Yarış Koşulu Var mı? (Mailbox Busy-Wait Race Condition)

[EN] Message-passing with mailboxes: full mailbox or empty receive returns error. Process retries until success. Does this lead to race conditions?

[TR] Hata kodu ile tekrar dene döngüsü yarış koşuluna yol açar mı?



Hayır, yarış koşuluna yol açmaz — ama başka sorunları vardır. Posta kutusu (mailbox) kendisi atomik gönder/al işlemleriyle korunuyorsa, birden fazla gönderici/alıcı olsa bile mesajlar kaybolmaz. Her mesaj yalnızca bir alıcıya gider.

Sorun: Busy-waiting (aktif bekleme) — süreç sürekli tekrar dener → CPU israfı ve livelock riski (hiç başarılı olamayan tekrar döngüsü).

E — Zamanlama (Scheduling) — Algoritmalar ve Hesaplamalar

Zamanlama (Scheduling) temel kavramları:

- Turnaround Time: İşin gönderilmesinden tamamlanmasına geçen toplam süre.
- Response Time: İstek yapılmasından ilk yanıt kadar geçen süre.
- Throughput: Birim zamanda tamamlanan iş sayısı.
- CPU Utilization: CPU'nun meşgul olduğu zaman oranı.
- Fairness: Her sürece adil CPU payı.

Soru 35 — CDC 6600 Processor Sharing — n Süreç ile Tamamlanma Süresi

[EN] CDC 6600 used round-robin at instruction level (zero overhead). If a process needs T sec alone, how much time with n processes sharing?

[TR] Her komut farklı süreçten alan sıfır-maliyetli processor sharing'de n süreç için süre?

Processor sharing: Her süreç CPU'nun $1/n$ 'ini alır

Tek başına çalışsaydı T saniye sürerdi.

n süreç paylaşıırken her süreç CPU'nun $1/n$ kadarını alır

→ Etkin hız = $1/n$ (n kat yavaşlar)

Tamamlanma süresi = $T \times n$

Örnek: $T=10$ sn, $n=5$ süreç → Her süreç 50 saniyede tamamlanır.



Processor sharing'de tüm süreçler eşit oranda yavaşlar. Her biri $T \times n$ sürede biter. Preemptive round-robin ile benzerdir ancak kuantum boyutu 1 komuttur.

Soru 37 — Round-Robin Listede Tekrar Eden Süreç (Duplicate in RR Queue)

[EN] Round-robin schedulers normally have each process once in the list. What would happen if a process occurred twice? Can you think of any reason for this?

[TR] Bir süreç Round-Robin kuyruğunda iki kez geçseydi ne olurdu? Neden yapılabilir?

Ne olur: Süreç diğer süreçlere göre iki kat CPU payı alır. Her tur tamamlandığında o süreç iki kez seçilir; bant genişliği 2x artar.

i Neden yapılabilir: Basit öncelik mekanizması olarak. Yüksek öncelikli süreci listeye 2x, 3x veya daha fazla eklemek etkin öncelik verir. Weighted Round-Robin'in ilkel biçimidir.

Soru 38 — CPU-Bound vs I/O-Bound Belirleme (Identifying CPU/IO-Bound)

[EN] Can you determine from source code whether a process is CPU-bound or I/O-bound? How can this be determined at run time?

[TR] Sürecin CPU-yoğun mu yoksa G/Ç-yoğun mu olduğu kaynak koddan anlaşılabilir mi?

Yöntem	Açıklama	Güvenilirlik
Kaynak Kod Analizi (Static Analysis)	G/Ç sistemi çağrıları (read, write, recv) aranır. Ama döngü sayısı, veri miktarı ve sistem yükü çalışma zamanında değişir.	Orta — kısmi bilgi
Çalışma Zamanı Ölçüm (Run-Time Profiling)	OS, her süreç için CPU kullanım süresini ve G/Ç bekleme süresini ölçer. Çok G/Ç bekleme → G/Ç yoğun. Az G/Ç → CPU yoğun.	Yüksek — kesin ölçüm
Zamanlayıcı Gözlemi (Scheduler Observation)	Süreç time quantum dolmadan bloke oluyorsa → G/Ç yoğun. Her seferinde tam kuantum kullanıyorsa → CPU yoğun.	Yüksek — pratik

Soru 39 — Önemli Sürecin Zamanlama Kararına Katılımı (Process-Assisted Scheduling)

[EN] Sometimes scheduling could be improved if an important process could play a role in selecting the next process. Give a situation where this could be used.

[TR] Önemli bir sürecin bloke olurken bir sonraki süreci seçmesine yardım ettiği durum?



Örnek — Veritabanı Sunucusu: Veritabanı işlem yöneticisi (transaction manager) bir kilidi serbest bırakırken, o kilidi bekleyen süreci doğrudan bilir. OS'ye 'kilidi bekleyen şu süreci çalıştır' ipucu vererek gereksiz context switch ve polling önlenir. Bu scheduler activation ve priority donation kavramlarına benzer bir yaklaşımdır.

Soru 40 — Kuantum Boyutu ve Bağlam Değişimi (Quantum Size vs Context Switch Time)

[EN] Explain how time quantum value and context switching time affect each other in round-robin scheduling.

[TR] Round-Robin'de kuantum boyutu ve bağlam değişim süresi birbirini nasıl etkiler?

Durum	Etki	Sonuç
Q çok küçük ($Q \ll S$)	Bağlam değişimi CPU süresinin büyük bölümünü yer	CPU verimliliği düşer; sistem zamanı boşa harcanır
Q çok büyük ($Q \gg S$)	Bağlam değişimi maliyeti ihmal edilebilir	Yüksek verimlilik ama yanıt süresi (response time) kötüleşir
$Q = S$	CPU zamanının yarısı bağlam değişimine gider	Verimsiz — pratik kullanılmaz
İdeal Q	$Q \gg S$ ve $Q \approx$ ortalama CPU burst	Hem verimli hem de iyi yanıt süresi



Tipik değerler: Bağlam değişimi $\approx 1-10 \mu s$, Kuantum $\approx 10-100 ms$. Bu oran bağlam değişim maliyetini $\sim 1\%$ düzeyinde tutar.

Soru 41 — Round-Robin CPU Verimliliği Formülleri (RR Efficiency Formulas)

[EN] Average process runs T before blocking on I/O. Context switch costs S . Round-robin quantum Q . Give CPU efficiency formula for each case.

[TR] Ortalama G/Ç'ye kadar çalışma T , bağlam değişimi S , kuantum Q . Her durum için CPU verimliliği formülü nedir?

Durum	Formül	Açıklama
(a) $Q = \infty$ (Sonsuz kuantum)	Verimlilik = $T / (T + S)$	Süreç bloke olana kadar çalışır; bağlam değişimi yalnızca G/Ç'de
(b) $Q > T$	Verimlilik = $T / (T + S)$	Süreç G/Ç'den önce kuantumunu kullanamaz; (a) ile aynı
(c) $S < Q < T$	Verimlilik = $Q / (Q + S)$	Her kuantumda bir bağlam değişimi; Q arttıkça verimlilik artar
(d) $Q = S$	Verimlilik = $Q / (Q + S) = S / 2S = 1/2 = \%50$	CPU zamanının yarısı bağlam değişimine gider

(e) $Q \rightarrow 0$	Verimlilik $\rightarrow 0$	Neredeyse tüm zaman bağlam değişiminde harcanır
-----------------------	----------------------------	---

i

Genel formül: Her kez Q süre çalışıp S süre bağlam değişimi yapıyorsa,
CPU Verimliliği = $Q / (Q + S)$
 $Q \gg S$ olduğunda verimlilik $\rightarrow 1$ (%100)

Soru 42 — Ortalama Yanıt Süresini Minimize Etme — SJF (Minimize Average Response Time)

[EN] Five jobs waiting: run times 9, 6, 3, 5, and X. In what order should they run to minimize average response time?

[TR] 5 işin bekleme süreleri: 9, 6, 3, 5, X. Ortalama yanıt süresini minimize eden sıra nedir?

i

Shortest Job First (SJF) ortalama yanıt süresini minimize eder. X değerine göre X'in konumu değişir:

X değeri	Sıralama	Ortalama Tamamlanma Süresi
$X \leq 3$	X, 3, 5, 6, 9	$(X)/5 + (X+3)/5 + (X+8)/5 + (X+14)/5 + (X+23)/5 = (5X+48)/5$
$3 < X \leq 5$	3, X, 5, 6, 9	$(3+X+3+X+8+...)/5$ — X, 3 ile 5 arasına girer
$5 < X \leq 6$	3, 5, X, 6, 9	X, 5 ile 6 arasına girer
$6 < X \leq 9$	3, 5, 6, X, 9	X, 6 ile 9 arasına girer
$X > 9$	3, 5, 6, 9, X	En uzun iş en sona

Örnek: X=4 için sıra: 3, 4, 5, 6, 9

Tamamlanma zamanları: 3, 7, 12, 18, 27

Ortalama = $(3+7+12+18+27)/5 = 67/5 = 13.4$ birim

Soru 43 — 5 Toplu İş — Zamanlama Algoritmaları Karşılaştırması (Batch Scheduling)

[EN] Jobs A(10 min, pri=3), B(6, pri=5), C(2, pri=2), D(4, pri=1), E(8, pri=4). Compute mean turnaround time for: (a) RR, (b) Priority, (c) FCFS, (d) SJF.

[TR] 5 toplu iş için ortalama tamamlanma süresini hesaplayın.

(a) Round Robin — Eş Zamanlı Paylaşım

i

RR'de tüm işler eşzamanlı çalışır. Her iş CPU payının eşit bölümünü alır. Toplam CPU = $10+6+2+4+8 = 30$ dakika. Her iş kendi süresinde biter (ancak son iş en geç biter çünkü başkaları önceden tamamlanır).
Ortalama turnaround = $(10+6+2+4+8) \times \text{belirli katsayı}$ ile yaklaşık hesaplanır.
Basit yaklaşım: tüm işler 30 dk noktasında biter $\rightarrow$ ortalama ≈ 30 dakika.

(b) Priority Scheduling (5 en yüksek)

Öncelik sırası: B(5), E(4), A(3), C(2), D(1)

B tamamlanır: $t = 6 \rightarrow \text{Turnaround B} = 6$

E tamamlanır: $t = 14 \rightarrow \text{Turnaround E} = 14$

A tamamlanır: $t = 24 \rightarrow \text{Turnaround A} = 24$

C tamamlanır: $t = 26 \rightarrow \text{Turnaround C} = 26$

D tamamlanır: $t = 30 \rightarrow \text{Turnaround D} = 30$

Ortalama = $(6+14+24+26+30) / 5 = 100/5 = 20$ dakika

(c) FCFS — Geliş Sırası: A, B, C, D, E

A: $t=10$ B: $t=16$ C: $t=18$ D: $t=22$ E: $t=30$

Ortalama = $(10+16+18+22+30) / 5 = 96/5 = 19.2$ dakika

(d) Shortest Job First (SJF)

Kısa $\rightarrow$ uzun sırası: C(2), D(4), B(6), E(8), A(10)

C tamamlanır: $t = 2 \rightarrow \text{Turnaround C} = 2$

D tamamlanır: $t = 6 \rightarrow \text{Turnaround D} = 6$

B tamamlanır: $t = 12 \rightarrow \text{Turnaround B} = 12$

E tamamlanır: $t = 20 \rightarrow \text{Turnaround E} = 20$

A tamamlanır: $t = 30 \rightarrow \text{Turnaround A} = 30$

Ortalama = $(2+6+12+20+30) / 5 = 70/5 = 14$ dakika $\leftarrow$ En iyi!

Algoritma	Ortalama Turnaround	Değerlendirme
Round Robin	~30 dakika	Adil ama en kötü ortalama
Priority	20 dakika	Orta; öncelik tersine dönme riski
FCFS	19.2 dakika	Basit; convoy effect riski
SJF	14 dakika ✓	En iyi ortalama — ancak açlık (starvation) riski

Soru 44 — CTSS — Kaç Kez Swap? (CTSS Swapping Count)

[EN] A process on CTSS needs 30 quanta to complete. How many times must it be swapped in, including the very first time?

[TR] CTSS'de 30 kuantum gerektiren süreç kaç kez swap in edilir (ilk kez dahil)?

i CTSS Hatırlama: CTSS (Compatible Time-Sharing System) geometrik sıra kullanan çok seviyeli kuyruklara sahiptir. Kuantum büyüklükleri: 1, 2, 4, 8, 16 ... kuantum. Her seviyede bir çalıştırma yapılır.

CTSS kuantum seviyeleri: 1, 2, 4, 8, 16 kuantum

1. swap in $\rightarrow$ 1 kuantum çalışır $\rightarrow$ kalan: 29

2. swap in → 2 kuantum çalışır → kalan: 27
3. swap in → 4 kuantum çalışır → kalan: 23
4. swap in → 8 kuantum çalışır → kalan: 15
5. swap in → 16 kuantum çalışır → kalan: -1 (tamamlandı!)

Toplam swap in sayısı = 5 (ilk kez dahil)

Soru 45 — CTSS Öncelik Sistemi — Boş Satır Saldırısı (CTSS Priority Gaming)

[EN] Can you think of a way to save the CTSS priority system from being fooled by random carriage returns?

[TR] Boş satır (carriage return) göndererek öncelik kazanma nasıl önlenir?

Problem: Kullanıcı boş satırlar göndererek çok G/Ç yaptığı izlenimi yaratır → sistem onu interaktif zannederek yüksek önceliğe alır.

Çözümler:



- ① Karakter sayma: Yalnızca anlamsız girişler (tek CR) tekrar eden süreçlerin önceliğini artırma.
- ② Gerçek çıktı üretimi izleme: Girdi verilen ama çıktı üretmeyen süreçler interaktif sayılmaz.
- ③ CPU kullanım ortalaması: Son N dönemdeki CPU kullanımına göre öncelik belirlenir; G/Ç miktarı tek başına kriter olmaz.

Soru 46 — Gerçek Zamanlı Zamanlama — Schedulable? (Real-Time Schedulability Test)

[EN] Two voice calls: period 5 msec, CPU 1 msec each. One video: period 33 msec, CPU 11 msec. Is this system schedulable?

[TR] 2 ses çağrısı (T=5ms, C=1ms) + 1 video (T=33ms, C=11ms). Zamanlanabilir mi?

Schedulability testi (Rate Monotonic): $\sum(C_i/T_i) \leq 1$

Ses 1: $C_1/T_1 = 1/5 = 0.200$

Ses 2: $C_2/T_2 = 1/5 = 0.200$

Video: $C_3/T_3 = 11/33 \approx 0.333$

Toplam = $0.200 + 0.200 + 0.333 = 0.733$

$0.733 \leq 1.0 \rightarrow$ SİSTEM ZAMANLANABİLİR (Schedulable)



CPU kullanım oranı %73.3 — limit %100'ün altında. Tüm periyodik görevler son tarihlerine (deadline) yetişebilir.

Soru 47 — İkinci Video Akışı Eklenebilir mi? (Adding Another Video Stream)

[EN] For the above problem, can another video stream be added and have the system still be schedulable?

[TR] Aynı sisteme ikinci bir video akışı (T=33ms, C=11ms) eklenebilir mi?

Önceki toplam: 0.733

Yeni video: $C/T = 11/33 \approx 0.333$

Yeni toplam = $0.733 + 0.333 = 1.066$

$1.066 > 1.0 \rightarrow$ SİSTEM ZAMANLANAMAYABİLİR (Not schedulable)

X

İkinci video akışı eklendiğinde toplam CPU kullanımı %106.6'ya çıkar. Bu 1.0 üzerinde olduğundan sistem zamanlanamaz; en az bir görev kendi son tarihini kaçıır.

Soru 48 — Yaşlandırma Algoritması — Sonraki Tahmin (Aging Algorithm Prediction)

[EN] Aging with $a = 1/2$. Previous runs (oldest to newest): 40, 20, 40, 15 msec. What is the prediction for the next time?

[TR] $a=1/2$ yaşlandırma ile 40, 20, 40, 15 ms geçmişten bir sonraki tahmin nedir?

Yaşlandırma formülü: $T_n = a \times T_{(n-1)} + (1-a) \times T_{(ger)}$

$a = 1/2$, $(1-a) = 1/2$

Başlangıç tahmini (T_0) = ilk ölçüm = 40

Adım 1 (gözlem=20): $T_1 = (1/2) \times 40 + (1/2) \times 20 = 20 + 10 = 30$

Adım 2 (gözlem=40): $T_2 = (1/2) \times 30 + (1/2) \times 40 = 15 + 20 = 35$

Adım 3 (gözlem=15): $T_3 = (1/2) \times 35 + (1/2) \times 15 = 17.5 + 7.5 = 25$

Bir sonraki çalışma için tahmin = 25 ms

Aging Algorithm	Yaşlandırma algoritması — üstel ağırlıklı ortalama
Exponential Averaging	Yeni ölçümlere daha fazla ağırlık verilir

Soru 49 — Yumuşak Gerçek Zamanlı — Maksimum x (Soft Real-Time Max x)

[EN] Soft real-time system: events with periods 50, 100, 200, 250 msec require 35, 20, 10, x msec CPU. What is the largest x for which the system is schedulable?

[TR] 4 periyodik olay için zamanlanabilir en büyük x değeri nedir?

Schedulability: $\sum(C_i/T_i) \leq 1$

$$35/50 + 20/100 + 10/200 + x/250 \leq 1$$

$$0.700 + 0.200 + 0.050 + x/250 \leq 1$$

$$0.950 + x/250 \leq 1$$

$$x/250 \leq 0.050$$

$$x \leq 0.050 \times 250 = 12.5 \text{ msec}$$

Maksimum $x = 12.5 \text{ msec}$

✓ $x \leq 12.5 \text{ ms}$ olduğunda toplam CPU kullanımı %100'e eşit olur. $x = 12 \text{ ms}$ gibi tam sayı alınabilir ($\%0.95 + \%4.8 = \%99.8 < \%100$).

Soru 50 — İki Seviyeli Zamanlama — Avantajları (Two-Level Scheduling)

[EN] Explain why two-level scheduling is commonly used. What advantages does it have over single-level scheduling?

[TR] İki seviyeli zamanlama neden yaygın kullanılır? Tek seviyeye göre avantajları?

Seviye	Görev	Açıklama
Üst Seviye (High-Level / Long-Term)	Hangi işler belleğe yüklenir?	Çok uzun vadeli karar. İş gönderim oranını kontrol eder. Yavaş çalışır (dakikalar-saatler arası).
Alt Seviye (Low-Level / Short-Term)	Hazır kuyruğundan hangisi CPU alır?	Milisaniyeler arası hızlı karar. Round-robin, priority gibi algoritma kullanır.

Avantajlar:

- ① Bellek basıncını kontrol: Çok fazla aktif süreç thrashing'e (sürekli sayfa hatası) neden olur. Üst seviye bu sayıyı sınırlar.
- ✓ ② Verimlilik: Alt seviye zamanlayıcı yalnızca bellekteki hazır süreçleri değerlendirir; diskten getirilmesi gereken süreçleri beklemez.
- ③ Adalet: Hem kısa hem uzun vadeli adalet sağlanabilir.

Soru 51 — Gerçek Zamanlı — 2 Ses + 1 Video (25 fps) Zamanlanabilir mi?

[EN] Two voice calls: period 5 msec, 1 msec CPU each. One video: 25 fps (period 40 msec), 20 msec CPU per frame. Is this system schedulable?

[TR] 2 ses (T=5ms, C=1ms) + video 25fps (T=40ms, C=20ms). Zamanlanabilir mi?

Ses 1: $C1/T1 = 1/5 = 0.200$
Ses 2: $C2/T2 = 1/5 = 0.200$
Video: $C3/T3 = 20/40 = 0.500$
Toplam = $0.200 + 0.200 + 0.500 = 0.900$
 $0.900 \leq 1.0 \rightarrow$ SİSTEM ZAMANLANABİLİR ✓

✓ CPU kullanımı %90. Sistem zamanlanabilir; her görev kendi döneminde CPU ihtiyacını karşılayabilir.

Soru 52 — Kernel Thread Zamanlaması — Politika/Mekanizma Ayrımı (Policy/Mechanism Separation)

[EN] Consider a system where it is desired to separate policy and mechanism for the scheduling of kernel threads. Propose a means of achieving this goal.

[TR] Kernel thread zamanlamasında politika ve mekanizmanın ayrılması nasıl sağlanır?

Katman	Sorumluluk	Örnek
Mekanizma (Mechanism — Kernel)	Bağlam değişimi, kuyruk ekleme/çıkarma, timer interrupt işleme	run_queue_enqueue(), context_switch(), preempt()
Politika (Policy — User Space)	Hangi iş yüksek öncelikli? Kuantum ne kadar? SJF mi RR mi?	Kullanıcı alanı scheduler daemon veya sistem çağrısı ile kernel'e bildirilir



Gerçek örnek: Linux'ta CFS (Completely Fair Scheduler) politikasını SCHED_NORMAL, SCHED_FIFO, SCHED_RR gibi sched_setscheduler() ile kullanıcı alanından değiştirebilirsiniz. Mekanizma (context switch kodu) değişmez; yalnızca politika güncellenir.

F — Açık Uçlu Quiz Soruları ve Cevapları

◆ Aşağıdaki sorular sınav hazırlığı içindir. Kısa, öz yanıtlar veriniz.

Q1: Süreç (process) ile thread arasındaki temel fark nedir? Hangi kaynaklar paylaşılır?

Cevap: Süreç bağımsız adres alanına sahipken thread'ler aynı sürecin belleğini, dosya tanımlayıcılarını ve diğer OS kaynaklarını paylaşır. Thread'lerin kendine özgü olan tek şey: kayıt seti (PC, SP, yazmaçlar) ve yığın (stack). Bu nedenle thread oluşturmak süreçten çok daha hızlı ve ucuzdur.

Q2: Busy-waiting (aktif bekleme) nedir ve neden kötüdür? Hangi alternatifleri vardır?

Cevap: Busy-waiting: bir koşul gerçekleşene kadar sürecin sürekli döngüde kontrol yapması (spin-lock). CPU zamanı boşa harcanır. Alternatifler: mutex (lock/unlock), semafor, koşul değişkeni (condition variable) — bunlar süreci bloke eder, kaynak hazır olduğunda uyandırır.

Q3: Çoklu programlama (multiprogramming) olmadan CPU kullanımı neden düşer?

Cevap: Tek program G/Ç beklerken (disk, ağ) CPU boşta kalır. Çoklu programlamada G/Ç bekleyen süreç bloke edilir ve CPU başka sürece verilir. $p = 50\%$ G/Ç bekleme ile 4 program: CPU kullanımı = $1 - 0.5^4 = 93.75\%$. Tek program: 50% .

Q4: SJF (Shortest Job First) neden ortalama bekleme süresini minimize eder? Dezavantajları nelerdir?

Cevap: SJF kısa işleri önce çalıştırarak onların bekleme süresini sıfıra yakın tutar. Uzun işler biraz daha bekler ama kısa işlerin kazanımı daha büyüktür — matematiksel olarak kanıtlanmıştır. Dezavantajlar: iş sürelerini önceden bilmek gerekir (pratikte zor), uzun işler aç kalabilir (starvation).

Q5: Rate Monotonic Scheduling (RMS) nedir? Schedulability testi nasıl yapılır?

Cevap: RMS: periyodik gerçek zamanlı görevleri kısa periyodludan uzun periyodluya öncelik vererek zamanlar. Schedulability testi: $\sum (C_i/T_i) \leq 1$. C_i = görevin CPU ihtiyacı, T_i = periyot. Bu koşul sağlanırsa tüm görevler kendi dead-line'larını karşılar.

Q6: Round-Robin'de kuantum boyutunun etkileri nelerdir? İdeal kuantum nasıl seçilir?

Cevap: Küçük kuantum: iyi yanıt süresi ama yüksek bağlam değişimi maliyeti → CPU verimsizliği. Büyük kuantum: FCFS'ye yaklaşır, yanıt süresi kötüleşir. İdeal: $Q \gg S$ (bağlam değişim süresi) ve $Q \approx$ ortalama CPU burst. Tipik: $Q = 10-100$ ms, $S = 1-10$ μ s → verimlilik $> 99\%$.

Q7: Priority inversion (öncelik tersine dönme) nedir? Mars Pathfinder'da bu sorun nasıl yaşandı?

Cevap: Yüksek öncelikli görev, düşük öncelikli görevin tuttuğu kilidi bekler; arada orta öncelikli görevler çalışabilir → yüksek öncelikli görev gecikmek zorunda kalır. Mars Pathfinder'da: H-öncelikli görev, L-öncelikli görevin mutex'ini bekledi. M-öncelikli görevler L'nin çalışmasını engelledi → H sonsuza bekledi. Çözüm: Priority Inheritance Protocol (PIP).

Q8: Kullanıcı düzeyi thread'lerin multicore CPU'dan yararlanamamasının nedeni nedir?

Cevap: Kernel yalnızca süreci görür ve tek bir CPU'ya atar. Kullanıcı düzeyi thread'ler kütüphane tarafından multipleks edilir ama hepsi aynı süreçte çalışır. Kernel birden fazla CPU'ya aynı süreci atamaz → paralel CPU kullanımı mümkün değil. Çözüm: kernel-level threads (pthreads) ile her thread farklı CPU'da çalışabilir.
